keep the master branch clean first

```
touch index.html
git add index.html
git commit -m "index commit" index.html
git status
git branch --> it will show the branch master, default branch
```

if you commit atleast 1 file, it will show the master branch

Its not good to work everyone on master branch, individual developer create their own branch

dev1branch
==========

```
git branch dev1branch  --> it will create a new branch dev1branch from master
git branch --> to list the branches, * represent the current branch
git checkout dev1branch --> to switch branch
git branch
touch dev1file{1..5}
git add dev1file*
git commit -m "dev1" dev1file*
```

dev1branch has 1 + 5 files = 6 (it got 1 file from master)

dev2branch
==========

```
git branch dev2branch --> this will create a new branch dev2branch from
dev1branch as we are now in dev1branch
git branch
git checkout dev2branch
git branch
touch dev2file{1..5}
git add dev2*
git commit -m "dev2 commit" dev2*
```

dev2branch has 1 + 5 + 5 = 11

dev3branch
==========

```
git branch
git branch dev3branch --> this will create a new branch from dev2branch
git checkout dev3branch
              or
              git checkout -b dev3branch --> this will also create and checkout
branch dev3branch, 2 commands in 1 shot
git branch
touch dev3file{1..5}
git add dev3*
git commit -m "dev3 commit" dev3*
```

dev3branch has 11 + 5= 16

dev4branch
==========

Now i want to create a new branch from master not from dev3

Now I want to checkout from master not from dev3 , so first checkout to master and create a branch, it will get only

```
git branch
git checkout master
ls
git branch dev4branch
```

```
git checkout dev4branch
ls

ls --> it should list only index.html
touch dev4file{1..5}
git add dev4*
git commit -m "dev4 commit" dev4*

dev4branch has 1 + 5 = 6


**** rename branch

git branch -m dev5branch devnewbranch  [renaming dev5branch to devnewbranch]

=======================
GIT MERGE - Merge between 2 branches
=======================

git branch
be in master

git checkout master

git merge dev1branch  --- what ever we have files in dev1branch will come to
master

Now push to GitHub, create a GitHub account
========================================
git branch

git remote add origin https://github.com/ReyazShaik/testrepo.git

git config --global credential.helper cache


git push origin master --> push the code of master to GitHub
username:
password: paste the token here
GitHub removed password authentication and put token system for better security
Generate token from GitHub
Profile --> settings --> developer settings --> personal access token -->
classic -->
Generate new token --> classic --> name:gitlearn --> select repo scopes -->
generate

Now push dev1branch files

git push origin dev1branch --> now see 2 branches in GitHub
git push origin dev2branch --> now see 3 branches in GitHub
git push origin dev3branch --> now see 4 branches in GitHub
git push origin dev4branch --> now see 5 branches in GitHub

=======================
GIT MERGE - Merge between 2 branches
=======================

git branch

git checkout dev2branch -- see the files

git checkout dev1branch -- see the files

git merge dev2branch  --- what ever we have files in dev2branch will come to
```

dev1branch

another command instead merge use rebase

git rebase dev4branch

Git Merge: Keeps the history intact and is safer for shared branches. Ideal for collaborative projects.
Git Rebase: Rewrites history for a cleaner, linear commit log. Best suited for private or feature branches.

======================
GIT REVERT -- accidental merges from one branch to another branch and undo those

git branch

git merge dev2branch

ls

git revert dev2branch --> this will delete the files from current branch which was merged
(don't to any changes to file, just quit the file)

========================
GIT BRANCH another example
========================

--> Create one test repo in GitHub

```
mkdir branchdemo
cd branchdemo
git init
touch python{1..10}

vi python1
This is python1
nice code. log code

git add .
git commit -m "pythonfiles" .

git branch

== what ever we are doing is in master branch
== A new developer came and said i will do some changes, is it good to do directly in master?
== create a new branch for developer

git branch developer
git branch
git checkout developer

vi python11
This is python11

vi python1
added new lines

git add .

git commit -m "pythonfilesnew added" .
```

```
== No impact on master branch
== Now lets go to master branch and see the data

git checkout master

ls

== Here we dont have python11 and python1 changes

if i want developer changes to master

git merge developer  [merging code from developer branch]

git remote add origin https://github.com/ReyazShaik/branching.git

Note: if you want to change remote origin " git remote set-url origin
https://github.com/ReyazShaik/branching.git&quot;

git push -u origin master

== Now see in GitHub, but you can see only one branch in GitHub because we
pushed from master, GitHub dosent know developer branch yet

git checkout developer
vi python1
added worst code
version 1.2.3

git add .

git commit -m "pythonfilesnew added" .

git push origin developer [we need to push from developer branch]

== Now in GitHub, you get Compare and Pull Request --> Merge Request
== After merging master branch contains the code of Developer branch

In your terminal, if you want to get latest code from GitHub

git checkout master

git pull


git merge
========

git checkout developer
vi python1
version 1.2.3.4

git add .
git commit -m "1.2.3.4" python1

git checkout master
vi python1
version 1.2.3.4.5

git add .
git commit -m "1.2.3.4.5" python1

git merge developer
-- merge conflit
```

vi python1

remove which ever you need, we can commit only 1
save it

git add python1
git commit --> no need to put filename

merge file will come , just wq!

git status

==========
GIT REBASE - same as MERGE
==========

git rebase dev3branch -- now you see dev3 also in dev1branch

==========

MERGE vs REBASE

Merge for public repos, rebase for Private
Merge stores history, rebase will not store the entire history (commits)
merge will show files, rebase will not show files